

Day 2 lab: MicroPython, sensor input, and a motion dataset

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 2, 2026

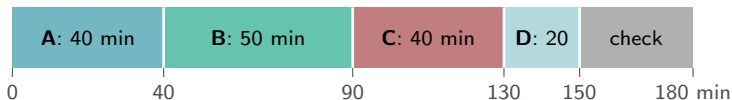
- ▶ **Goal:** your group has a labelled motion dataset that Day 3 uses, with a constant sampling rate and a correct split between training and test data.
- ▶ **Board:** the XIAOML Kit. Each group has two students and one kit.
- ▶ **Deliverable:** the folder `data/` with your recordings and the file `split.json`, and the file `report.md` with the data card and the Decision Log.

From the lecture to the lab

The lecture gave the rules: a timer for a constant rate, a window that holds the pattern, and a split by session. In the lab, you measure your rate, you record four motion classes, and you measure the effect of a wrong split.

Key point: Day 3 trains a model on the dataset of today. An error in the data today gives a wrong accuracy tomorrow.

Plan of the three hours



Part	Work	Time
A	MicroPython start: firmware, REPL, GPIO, timer, I2C scan	40 min
B	Sensor input: the IMU at a fixed rate, serial port and Wi-Fi	50 min
C	Dataset: logger, four motion classes, split by session	40 min
D	Edge Impulse: upload and inspect the same data	20 min
Check by the instructor		30 min

Write each result in `report.md` when you get it. Do not wait for the end of the lab.

Hardware for each group

- ▶ XIAOML Kit: the XIAO ESP32S3 Sense on the expansion board with the IMU.
- ▶ USB-C data cable.
- ▶ Wi-Fi antenna of the kit, for the last step of Part B.
- ▶ Laptop in the lab Wi-Fi.

Software

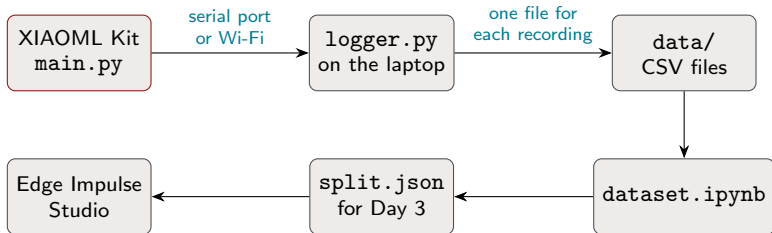
- ▶ MicroPython firmware v1.29.0, esptool, and mpremote.
- ▶ Python 3.10 or later with numpy, matplotlib, pyserial, and jupyterlab.
- ▶ An Edge Impulse account. The CLI is optional.
- ▶ Arduino IDE 2, for the last step.

Lab files Labs/day02/. Run all commands from this folder.

Setup steps Labs/SETUP.md, steps 5 and 6

Key point: MicroPython replaces the Arduino firmware of the board. The last step of the lab puts the Arduino firmware back. Day 3 needs it.

The path of the data



- ▶ **Part A:** MicroPython runs on the kit.
- ▶ **Part B:** the kit sends 50 IMU samples each second. You check the rate with `host/check_rate.py`.
- ▶ **Part C:** the logger writes the recordings. The notebook checks them and writes the split.
- ▶ **Part D:** you upload the same files to Edge Impulse Studio.

Part A: write the MicroPython firmware (40 min)

1. Download the firmware file and check it.
2. Start bootloader mode: disconnect the USB cable, hold the B (boot) button of the XIAO, connect the cable, release the button.
3. Erase the flash and write the firmware.
4. Press the RST button of the expansion board.

```
bash ../hardware/HW-01/get_firmware.sh
python3 -m esptool --chip esp32s3 --port PORT erase_flash
python3 -m esptool --chip esp32s3 --port PORT write_flash 0 \
  ../hardware/HW-01/downloads/SEEED_XIAO_ESP32S3-20260824-v1.29.0.bin
```

- ▶ PORT is the serial port of the board: /dev/ttyACM0 (Linux), /dev/cu.usbmodem101 (macOS), or COM4 (Windows).
- ▶ The port name can change after the reset. The command `mpremote devs` lists the ports.
- ▶ The boot button is on the XIAO, not on the expansion board. It is very small.

```
mpremote connect PORT repl
```

```
>>> import sys, machine, gc
>>> sys.implementation
>>> machine.freq()
>>> gc.mem_free()

>>> from machine import Pin
>>> led = Pin(21, Pin.OUT)
>>> led.value(0)      # LED on
>>> led.value(1)      # LED off
```

- ▶ The REPL is the prompt of MicroPython. You enter one line and you see the result.
- ▶ **You record:** the MicroPython version, the clock frequency in Hz, and the free memory in bytes.
- ▶ **You see:** the LED of the XIAO is on when the pin is low.
- ▶ Press Ctrl-] to close the REPL.

Key point: Part 1 of the lecture gave the clock and the memory of the board. The REPL shows the two values that your program has.

```
mpremote connect PORT run board/blink_timer.py
```

The LED changes two times each second. Press the boot button.
ticks = ..., presses = ...

- ▶ A hardware timer calls a handler two times each second. The handler changes the LED and counts.
- ▶ A pin interrupt counts each press of the boot button.
- ▶ **You record:** the number of ticks after 10 s.

```
mpremote connect PORT run board/i2c_scan.py
```

```
Found 2 device(s)
0x3C  OLED display (SSD1306)
0x6A  IMU (LSM6DS3TR-C)
```

- ▶ **You record:** the two addresses.

Source: The two outputs show the form that the scripts print. Nobody measured them on the kit.

Part B: read the IMU (50 min)

```
mpremote connect PORT fs cp board/lsm6ds3.py :lsm6ds3.py
mpremote connect PORT repl
```

```
>>> from machine import I2C, Pin
>>> from lsm6ds3 import LSM6DS3
>>> imu = LSM6DS3(I2C(0, sda=Pin(5), scl=Pin(6), freq=400000))
>>> hex(imu.chip_id)           # the identity register: 0x6a
>>> imu.read()                 # (ax, ay, az, gx, gy, gz)
```

- ▶ The first command copies the driver to the file system of the board. The file stays there after a reset.
- ▶ `imu.read()` gives the acceleration in g and the angular rate in degrees each second.
- ▶ **You see:** with the kit flat on the table, one acceleration axis is near 1.0 g . The other two are near 0.
- ▶ **You record:** the six values.

Key point: The driver has about 130 lines. Read it: it uses the registers of Part 1 of the lecture.

Part B: predict the rate, then measure it

The first version of `board/imu_stream.py` reads the sensor, prints one line, and then waits 20 ms.

Prediction. Is the real rate equal to 50 Hz, lower, or higher? By how much? Write your prediction in the report before you measure.

```
mpremote connect PORT fs cp board/imu_stream.py :main.py
mpremote connect PORT reset
python3 host/check_rate.py --port PORT --seconds 20 --save rate_wait.csv
```

```
Samples:          ...
Lost lines:        ...
Mean rate:         ... Hz (nominal 50.00 Hz)
Period min / max:  ... us / ... us
RESULT: ...
```

- ▶ A script with the name `main.py` starts at each power-on.
- ▶ **You record:** the mean rate and the result.

Source: The output shows the form of the lines. Nobody measured the rate on the kit.

Part B: task B1, a deadline for each sample

Change the end of the loop in
board/imu_stream.py:

1. Start deadline with
`time.ticks_us()` before the
loop.
2. Add `PERIOD_US` with
`time.ticks_add()`.
3. Get the time to the deadline
with `time.ticks_diff()`.
4. Sleep only for this time.
5. If the time is negative, start a
new time base.
 - ▶ **You see:** RESULT: PASS - the sampling rate is constant.
 - ▶ **You record:** the mean rate, the minimum period, and the maximum period.
 - ▶ The check passes when the mean rate is within 1 percent of 50 Hz, at most 1 percent of the periods are outside 18 to 22 ms, and no line is lost.

```
# The first version: replace this line.  
time.sleep_us(PERIOD_US)
```

```
mpremote connect PORT fs cp \  
    board/imu_stream.py :main.py  
mpremote connect PORT reset  
python3 host/check_rate.py --port PORT \  
    --seconds 20 --save rate_deadline.csv
```

1. Connect the antenna to the XIAO.
2. Write the name of the lab Wi-Fi, its password, and the IP address of your laptop in `board/config.py`.
3. Copy the settings and start the Wi-Fi script. In a second terminal, measure the rate.

```
mpremote connect PORT fs cp board/config.py :config.py
mpremote connect PORT run board/imu_wifi.py

python3 host/check_rate.py --udp 5005 --seconds 20      # second terminal
```

- ▶ The board sends the lines of 5 samples in one UDP packet: 10 packets each second.
- ▶ UDP does not repeat a packet that the network loses. The sample number shows each lost sample.
- ▶ **You record:** the mean rate and the number of lost lines.
- ▶ **Question:** which connection loses samples, the serial port or Wi-Fi? Why?

Part C: the logger (40 min)

Open `host/logger.py`. Complete the two functions with the mark `TODO` (student).

Task	Function	Example
C1	<code>file_name</code>	label <code>lift</code> , session <code>s2</code> , number 7 gives <code>lift.s2.07.csv</code>
C2	<code>count_lost</code>	the sample numbers 4, 5, 6, 9, 10 give 2 lost samples

Test the two functions with no board:

```
python3 host/logger.py --self-test
```

```
Task C1 (file_name): complete
```

```
Task C2 (count_lost): complete
```

- ▶ The file name holds the label and the session. The notebook and Edge Impulse Studio read the label from the file name.
- ▶ The sample number of the board shows a lost sample. A recording with a lost sample is not a recording at 50 Hz.

Write the protocol in the report before the first recording: one sentence for each class, and the person of each session.

- ▶ idle: the kit lies on the table.
- ▶ terrestrial: left and right in a horizontal line.
- ▶ lift: up and down.
- ▶ maritime: slowly on all axes, as a boat on waves.
- ▶ Three sessions: s1, s2, s3.
- ▶ Change the person, the speed, or the size of the motion between the sessions.
- ▶ 4 recordings of 10 s for each class in each session.

```
python host/logger.py --port PORT --label lift --session s1 --person A \  
--count 4 --plot
```

```
Recording 1 of 4, class 'lift'. Start the motion now.  
recording 10 s ...  
saved data/lift.s1.01.csv: 500 samples, ... Hz, ... lost, ... clipped  
plot: data/plots/lift.s1.01.png
```

Look at each plot. If a plot does not agree with its label, delete the file and record again.

Source: The output shows the form of the lines. Nobody recorded with the kit.

Part C: the notebook has five tasks

Task	You write	The notebook checks
1	The mean sampling rate of a recording	Two known cases: 50 Hz and 41.67 Hz
2	The split by session	No session is in the two sets. Each class is in the two sets.
3	The number of windows of a recording	500 samples give 41 windows of 100 samples with a stride of 10
4	Your prediction for the leakage experiment	Not checked. You compare it with the result.
5	The data card in <code>report.md</code>	Not checked. The instructor reads it.

- ▶ Start the notebook with `jupyter lab dataset.ipynb`.
- ▶ The notebook writes `data/split.json`. Day 3 reads this file.
- ▶ If `data/` has no recordings, the notebook uses a fallback dataset with simulated signals. Use it only if your group has no recordings.

	Procedure A (wrong)	Procedure B (correct)
Step 1	Cut all recordings into windows	Split the recordings by session
Step 2	Normalize with the statistics of all windows	Cut the windows in each set
Step 3	Mix the windows: 80 percent training, 20 percent test	Normalize with the statistics of the training set

Prediction (task 4). Which procedure gives the higher test accuracy? Which two numbers do you expect? Write them before you run the experiment.

```
Procedure A, random split of the windows:    ... percent
Procedure B, test session s1                  ... percent
Procedure B, test session s2                  ... percent
Procedure B, test session s3                  ... percent
```

The classifier is the same in the two procedures: the nearest neighbour on six features of a window. Only the split is different.

```
python3 host/to_edge_impulse.py --data data
edge-impulse-uploader --category training ei_upload/training/*.csv
edge-impulse-uploader --category testing ei_upload/testing/*.csv
```

1. Convert: the time in milliseconds, the acceleration in m/s^2 , one folder for each set.
2. Make the project motion-gNN in Edge Impulse Studio, and upload. With no CLI, upload in the browser: Data acquisition, then Add data.
3. Do not use the automatic split of the Studio. It does not know your sessions.
4. Inspect: the four labels, one plot of each class, the duration of the training data and of the test data.

Before you leave: the Arduino firmware

In the Arduino IDE, set Tools > Erase All Flash Before Sketch Upload > Enabled. Upload Labs/day01/sketches/blink/blink.ino. Then set the option to Disabled again. If the upload fails, start bootloader mode first.

The instructor checks this at the end of the lab:

- ▶ ✓ `host/check_rate.py` prints `RESULT: PASS` for the deadline version. The report has the rate of the two versions.
- ▶ ✓ The report has the two predictions: the rate of Part B, and task 4 of the notebook. You wrote them before the measurement.
- ▶ ✓ `python3 host/logger.py --self-test` prints `complete` for the two tasks.
- ▶ ✓ The folder `data/` has four classes and a minimum of two sessions. The notebook reports no recording with a problem.
- ▶ ✓ The notebook prints `complete` for tasks 1, 2, and 3. No session is in the training set and also in the test set.
- ▶ ✓ The data card is complete: classes, duration, sampling rate, split, known limits.
- ▶ ✓ The Edge Impulse project shows the four classes.
- ▶ ✓ The board runs the Arduino Blink sketch again.

Hand in: the folder `data/` with `split.json`, and the file `report.md`.

Write about 100 words. State one design decision, give your measured numbers, and name the trade-off.

Your group has 10 more minutes to record data.

- ▶ Do you add more recordings to the three sessions that you have?
- ▶ Or do you record a new session with a different person?

Use the two results of your leakage experiment.

A good Decision Log:

- ▶ gives the accuracy of procedure A and the accuracies of procedure B from your notebook,
- ▶ says what the difference between the two numbers means for a new person,
- ▶ names what you lose with your decision. Example: variation against the quantity of data for each session.

If a part does not work

Problem	Fallback
MicroPython does not start on your kit	Upload the Arduino sketch <code>sketches/imu_data_collection</code> . It sends the same lines. Continue with the rate check and with Part C.
The board does not work at all	Start <code>python3 host/sim_board.py --label lift --session s1</code> . Use <code>--udp 5005</code> in place of <code>--port PORT</code> . The signals are simulated.
The lab Wi-Fi does not work	Do not do the Wi-Fi step of Part B. Write the reason in the report.
Your group has no dataset at the end	The notebook makes the fallback dataset. Write in the data card that the signals are simulated.

Key point: A fallback keeps your group in the lab. Write in the report which fallback you used.

Problem	Fix
esptool cannot connect	Start bootloader mode again. Hold the button before you connect the cable.
mpremote prints could not enter raw repl	main.py uses the port. Open the REPL, press Ctrl-C, then Ctrl-].
ImportError: no module named 'lsm6ds3'	Copy the driver to the board: mpremote connect PORT fs cp board/lsm6ds3.py :lsm6ds3.py
check_rate.py prints Not enough samples: 0	Check that main.py is on the board. Close the REPL and the Serial Monitor.
check_rate.py --udp gets no samples	Check the laptop address in config.py. Permit the UDP port 5005 in the firewall of the laptop.
The board does not connect to the Wi-Fi	Connect the antenna. Check config.py. The XIAO has 2.4 GHz Wi-Fi only.
The logger prints WARNING: samples are missing	Record again. Close other programs that use the port.

The README of the lab has the complete list.

This lecture adapts material from these sources:

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. mlsysbook.ai, CC BY-NC-SA 4.0.
- ▶ *TinyML Made Easy: XIAO ESP32S3* by Marcelo Rovai. github.com/Mjrovai/XIAO-ESP32S3-Sense, Apache-2.0.

Material from a source with the licence CC BY-NC-SA 4.0 keeps that licence.